Day (39).

(3) Transfer Flow control statements :-

=> The purpose of Transfer flow control statements in python is that "to change the control of PVM from one part of the program to another part of program".

=> In python programming, we have 4 types of Transfer flow control statements. They are:

(i) break
(ii) continue
(iii) pass
(iv) return

① break :-

→ break is a keyword.

→ break keyword must be used always inside of loops otherwise we get syntax error.

→ The purpose of break statement is that " to terminate the execution of loop logically when certain condition is satisfied and pvm control comes of corresponding loop and executes other statements in the program".

→ When break statement takes place inside for loop or while loop then pvm will not execute coreresponding else block (bcoz loop is not becoming false) but it executes other statements in the program.

syntax①

```
for varname in Iterable_object:
    if (test cond):
        break
```

syntax②

```
while (Test cond-1):
    if (test cond-2):
        break
```

program for demonstrating break stmts :-

ex①.

```
s="python"
print("By using for loop without Break stmts")
for ch in s:
    print(ch)
else:
    print("else part of for loop")
print("_ _ _ _ _ _")
```

my requirement is to display only PYT without using indexing

```
for ch in s:
    if(ch=="H"):
        break
    print(ch, end="")
```

```
else:
    print("else part of for loop")
print()
print("program execution completed!!")
```

(ek khaali line print krega) (ye nhi dena pe)

← ye print hoga ye.

Ex2

```
import sys
s="python"
print("By using while loop without break stmt")
i=0
while(i<len(s)):
    print(s[i])
    i=i+1
else:
    print("else part of while loop")
print("- - - - - -")
# my requirement is PYTH
i=0
while(i<len(s)):
    if(s[i]=="O"):
        break
    print(s[i],end="")
    i=i+1
else:
    print("else part of while loop")
print()
print("program execution completed")
```

this is O not zero

else: print("\t{}".format(s[i]))

program deciding whether the given number is prime or not.

Ex3.

```
n=int(input("Enter a number:"))
if(n<=1):
    print("{} is invalid input!!".format(n))
else:
```

Syntax ② :-

```
while (Test cond):
    if (Test cond):
        continue
    statement - 1        # writtn aftr continue statement
    statement - 2
    statement - n
    - - - - - - - -
    - - - -
```

Program for demonstrating Continue statement.

```
s = "pYTHoN"
For ch in s:
    print ("\t{}".format(ch))
else:
    print (" i am from else of for loop")
print ("programs execution completed")
print (" — —  — —  — — —")
# want to display PYHON
for ch in s:
    if (ch == "T"):
        continue
    print (ch)
else:
    print (" i am from else of for loop")
print ("program execution completed")
```

ex ②

```
s = "python"
# Display all the letters
i = 0
while (i < len(s)):
    print("\t{}".format(s[i]))
    i = i+1
else:
    print (" i am from else of while loop")
print (" program execution completed")
print (" — — — — — —").
```

```
#want to display PYTHON.
i=0
while (i<len(s)):
    if (s[i]=="T"):
        j=i+1
        continue
    else:
        print("\t{}".format(s[i]))
    i=i+1
else:
    print("i am from else of while loop")
print("program execution completed")
```

ex③

```
#want to display PTON.
if (s[i]=="P") or (s[i]=="H"):
```

Bas phi likhna hai.

ex④

```
if(s[i] in["P","H"]):
```

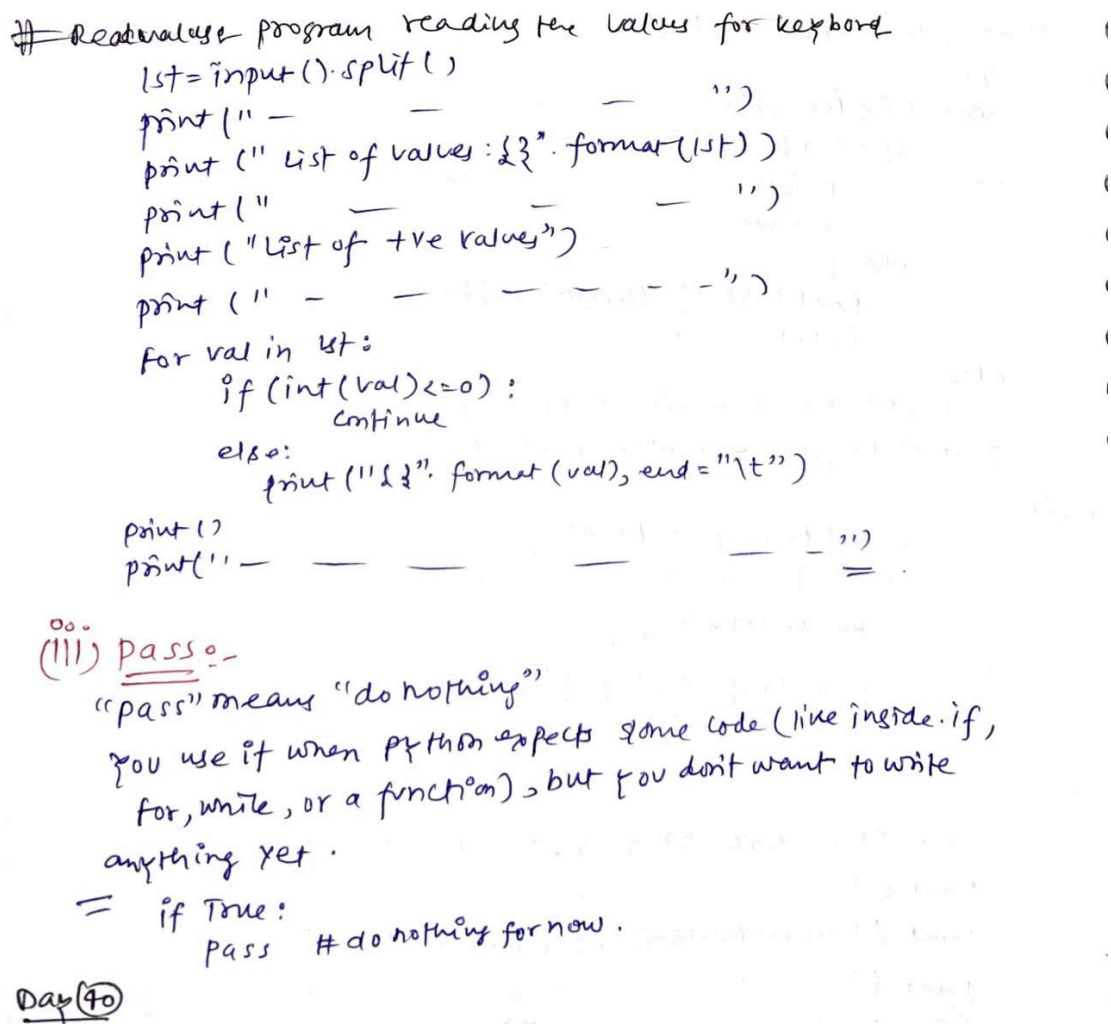
ex⑤

```
lst=[10,-34,56,-23,0,-2,45,67,-56,12]
print("          -          -          ")
print("given Numbers:{}".format(lst))
print("          -          -          ")
print("list of +ve values")
print("          -          -          ")
for val in lst:
    if(val<=0):
        continue
    print("\t{}".format(val))
print("          -          -          ")
```

ex⑥

Change the value "List of -ve values.

```
if(val>=0)
```



TRANSFER FLOW CONTROL STATEMENT:

- 1.break
- 2.continue
- 3.paas
- 4.return

1.break

- syntax 1:= for varname in iterableobject:
- ```
  if(test cond):
      break
  ```
- syntax 2:= while (testcond-1):
- ```
  if(test cond-2):
      break
  ```

In [7]:
```
#program for demonstrating break statements:
#ex1:
s="python"
print("by using for loop without break statements")
for ch in s:
    print(ch)
else:
```

```
    print("else part of for loop")
print("==================================")
#my rqrmnt is to display only pyt without using indexes
for  ch in s:
    if(ch=="h"):
        break
    print(ch,end="")
else:
    print("else part of for loop")
print()
print("program execution completed!!!")
```

```
by using for loop without break statements
p
y
t
h
o
n
else part of for loop
==================================
pyt
program execution completed!!!
```

In [19]:
```
#ex2:
s="python"
print("by using while loop without break stmnt")
i=0
while(i<len(s)):
    print(s[i])
    i=i+1
else:
    print("else part of while loop")
print("=========================================")
#my requirement is pyth
i=0
while(i<len(s)):
    if(s[i]=="o"):
        break
    print(s[i],end="")
    i=i+1
else:
    print("else part of while loop")
print()
print("program execution completed")
```

```
by using while loop without break stmnt
p
y
t
h
o
n
else part of while loop
=========================================
pyth
program execution completed
```

In [30]:
```
#### program for deciding whether the given number is prime or not:
#ex:-1
n=int(input("enter a number"))
```

```
if(n<=1):
    print("{}.invalid input".format(n))
else:
    res=True
    for i in range(2,n):
        if(n%i==0):
            res=False
            break
    if(res):
        print("{} is a prime number".format(n))
    else:
        print("{} is not a prime number".format(n))
```

```
7 is a prime number
```

In [2]:
```
#### program for deciding whether the given number is prime or not:
#ex:-2
n=int(input("enter a number"))
if(n<=1):
    print("{}.invalid input".format(n))
else:
    res="prime"
    for i in range(2,n):
        if(n%i==0):
            res="not prime"
            break
    if(res=="prime"):
        print("{} is {}".format(n,res))
    else:
        print("{} is {}".format(n,res))
```

```
7 is prime
```

2.continue:

- syntax 1:- for varname in iterable_object
- ```
            if(test cond):
               continue
          stmnt 1  #written after continue stmnt
          .......
          stmnt n
  ```
- syntax 2:- while(test cond):

- ```
        if (test cond):
             continue
           stmnt 1
            stmnt 2
           stmnt n
  ```

In [3]:
```
#program for demonstrating continue statements
#ex:1
s="python"
for ch in s:
    print("\t{}".format(ch))
else:
    print("i am from else of for loop")
```

```
print("program execution completed")
print("================================================================================")
#want to dispaly python
for ch in s:
    if(ch=="t"):
        continue
    print(ch)
else:
    print("i am from else of for loop")
print("program execution completed")
print("================================================================================")
```

```
       p
       y
       t
       h
       o
       n
i am from else of for loop
program execution completed
================================================================================
p
y
h
o
n
i am from else of for loop
program execution completed
================================================================================
```

In [5]:
```
#ex 2:-
s="python"
#display all the letter
i=0
while(i<len(s)):
    print("\t{}".format(s[i]))
    i=i+1
else:
    print("i am  from else of while loop")
print("program execution completed")
print("================================================================")
#want to display pyhon
i=0
while(i<len(s)):
    if(s[i]=="t"):
        i=i+1
        continue
    else:
        print("\t{}".format(s[i]))
        i=i+1
else:
    print("i am from else of while loop")
print("program execution completed")
```

```
        p
        y
        t
        h
        o
        n
i am  from else of while loop
program execution completed
============================================================
        p
        y
        h
        o
        n
i am from else of while loop
program execution completed
```

In [8]:
```
#ex3:-
s="python"
#display all the letter
i=0
while(i<len(s)):
    print("\t{}".format(s[i]))
    i=i+1
else:
    print("i am  from else of while loop")
print("program execution completed")
print("============================================================")
#want to display pton
i=0
while(i<len(s)):
    if(s[i]=="y") or (s[i]=="h"):
        i=i+1
        continue
    else:
        print("\t{}".format(s[i]))
        i=i+1
else:
    print("i am from else of while loop")
print("program execution completed")
```

```
        p
        y
        t
        h
        o
        n
i am  from else of while loop
program execution completed
============================================================
        p
        t
        o
        n
i am from else of while loop
program execution completed
```

In [10]:
```
#ex 4:-
s="python"
#display all the letter
```

```
i=0
while(i<len(s)):
    print("\t{}".format(s[i]))
    i=i+1
else:
    print("i am  from else of while loop")
print("program execution completed")
print("=================================================================")
#want to display pton
i=0
while(i<len(s)):
    if(s[i]in["y","h"]):
        i=i+1
        continue
    else:
        print("\t{}".format(s[i]))
        i=i+1
else:
    print("i am from else of while loop")
print("program execution completed")
```

```
        p
        y
        t
        h
        o
        n
i am  from else of while loop
program execution completed
=================================================================
        p
        t
        o
        n
i am from else of while loop
program execution completed
```

In [17]:
```
#ex 5:-
lst=[10,34,56,-23,0,-2,45,67,-56,12]
print("========================================")
print("given numbers:{}".format(lst))
print("========================================")
print("list of values")
print("========================================")
for val in lst:
    if (val<=0):
        continue
    print("\t{}".format(val))
print("========================================")
```

```
==========================================
given numbers:[10, 34, 56, -23, 0, -2, 45, 67, -56, 12]
==========================================
list of values
==========================================
        10
        34
        56
        45
        67
        12
==========================================
```

In [16]:
```
#ex 6:-
#ex 5:-
lst=[10,34,56,-23,0,-2,45,67,-56,12]
print("==========================================")
print("given numbers:{}".format(lst))
print("==========================================")
print("list of values")
print("==========================================")
for val in lst:
    if (val>=0):
        continue
    print("\t{}".format(val))
print("==========================================")
```

```
==========================================
given numbers:[10, 34, 56, -23, 0, -2, 45, 67, -56, 12]
==========================================
list of values
==========================================
        -23
        -2
        -56
==========================================
```

In [19]:
```
# reading program reading the values for keyboard
lst=input().split()
print("================================================")
print("list of values:{}".format(lst))
print("================================================")
print("list of +ve values")
print("================================================")
for val in lst:
    if(int(val)<=0):
        continue
    else:
        print("{}".format(val),end="\t")
print()
print("================================================")
```

```
================================================
list of values:['54', '-85', '45', '14', '2', '54', '-48', '-78']
================================================
list of +ve values
================================================
54      45      14      2       54
================================================
```

In [ ]: